Algorithm2: *Problem Solving* with *Algorithms* and *Data Structures*

Lecture 05.

동적 계획법 (2)

경북대학교 배준현 교수
(joonion@knu.ac.kr)

Lec. 05. 동적 계획법 (2)

최적화 문제: *Optimization Problem*

문제의 답이 하나가 아니라 여러 개(*candidate solutions*)이고, 그 중에서

어떤 기준(*criteria*)에 따라 **가장 좋은 답**(*optimal solution*)을 찾아 내는 문제

조합 문제:

n개의 원소 중에서 r개를 순서 없이 골라내는 **방법의 수**

최적화 문제:

n개의 사과 중에서 r개를 골라 무게의 합을 **최대화**하는 문제

가장 무거운 사과와 가장 가벼운 사과의 무게 차이를 **최소화**하는 문제

Lec. 05. 동적 계획법 (2)

최적화 문제를 푸는 여러 가지 방법

1. 완전 탐색으로 푸는 방법 (Chap. 6)
2. 동적 계획법으로 푸는 방법 (Chap. 8)
3. 조합탐색(백트래킹)으로 푸는 방법 (Chap. 11)
4. 결정 문제로 바꿔 해결하는 방법 (Chap. 12)

최적화 문제를 푸는 동적 계획법 레시피

1. 최적값(*optimal value*)을 찾는 동적계획법 알고리즘을 설계한다.
2. 최적값을 찾아가는 선택 과정을 별도의 저장 공간(path table)에 기록한다.
3. (주로 재귀 함수를 이용해) 선택 과정을 따라가며
   최적해(*optimal solution*)를 역추적하여 재구성한다.

예) 모든 쌍 최단 경로 문제: *Floyd* 알고리즘 (13강 (교재 30장)에서 다룸)

```
# Floyd-Warshall algorithm
for k from 1 to n
    for i from 1 to n
        for j from 1 to n
            if dist[i][j] > dist[i][k] + dist[k][j]:
                dist[i][j] = dist[i][k] + dist[k][j]
                path[i][j] = k
```

$dist[i][j]$**에 최단경로의 길이(최적값)을 저장**

$path[i][j]$**에 거쳐가는 중간 정점** k**를 저장**

$path$ **테이블로부터 임의의 정점쌍** (u, v)**의 최단 경로(최적해)를 재구성할 수 있다.**

삼각형 위의 최대 경로 문제: 최적값이 아니라 최적해를 찾고자 한다면?

삼각형 위의 최대 경로 문제는 최적화 문제:

최적값(*optimal value*): 경로에 포함된 숫자의 합 중에서 최댓값

최적해(*optimal solution*): 최적값을 가지는 경로

$T =$

6				
1	2			
3	7	4		
9	4	1	7	
2	7	5	9	4

$dp =$

28				
20	22			
19	18	20		
16	11	10	16	
2	7	5	9	4

최적값을 찾는 과정을 기록하기

$T =$

6				
1	2			
3	7	4		
9	4	1	7	
2	7	5	9	4

$dp =$

28				
20	22			
19	18	20		
16	11	10	16	
2	7	5	9	4

$path =$

+1				
-1	+1			
-1	-1	+1		
+1	-1	+1	-1	
0	0	0	0	0

$path[y][x] = -1$: **바로 아래로 이동**

$path[y][x] = +1$: **오른쪽 아래로 이동**

$path[y][x] = 0$: **양쪽 모두 가능**

Lec. 05. 동적 계획법 (2)

```python
# 최적해를 재구성할 수 있도록 path 테이블에 선택 경로 저장
def trianglepath(n, T):
    dp = T[n - 1][:]
    path = [[0] * (i + 1) for i in range(n)] # 선택 경로 저장을 위한 테이블

    for i in range(n - 2, -1, -1):
        for j in range(i + 1):
            down, right = dp[j], dp[j + 1]
            if down > right: # 바로 아래가 더 크면
                path[i][j] = -1 # 경로 테이블에 -1 저장
            elif down < right: # 오른쪽 아래가 더 크면
                path[i][j] = +1 # 경로 테이블에 +1 저장
            else:
                path[i][j] = 0 # 둘 다 같을 경우 0 저장

            dp[j] = T[i][j] + max(down, right)

    return dp[0], path # 최적값과 함께 경로 테이블 반환
```

경로 기록으로 최적해(최대합을 갖는 경로의 숫자들) 재구성하기

```
# 경로 테이블을 이용하여 최적해를 재구성한다.
def reconstruct(y, x):
    global n, T, path
    if y == n - 1: # 맨 아래쪽에 도착했다.
        return [T[y][x]]
    elif path[y][x] == -1: # 바로 아래로 내려간다.
        return [T[y][x]] + reconstruct(y + 1, x)
    elif path[y][x] == +1: # 오른쪽 아래로 내려간다.
        return [T[y][x]] + reconstruct(y + 1, x + 1)
    else: # path[y][x] == 0 # 어느 쪽이든 상관없다.
        return [T[y][x]] + reconstruct(y + 1, x + 1)
```

Lec. 05. 동적 계획법 (2)

```
'''
삼각형 위의 최대 경로 문제 최적해 재구성하기
'''
for _ in range(int(input())):
    n = int(input()) # 삼각형의 크기
    T = [list(map(int, input().split())) for _ in range(n)] # 삼각형의 숫자
    optval, path = trianglepath(n, T)
    optsol = reconstruct(0, 0)
    print(optval)
    print(*optsol)
```

AL2.5.1: 여행 짐 싸기 (ID: PACKING)

문제

여행을 떠나기 전날까지 절대 짐을 싸지 않는 버릇이 있는 재훈이는 오늘도 비행기 타기 전날에야 가방을 싸기 위해 자리에 앉았습니다. 비행기 규정상 재훈이는 캐리어를 하나만 가지고 갈 수 있는데, 아무래도 가져가고 싶은 물건들이 캐리어 안에 다 들어가지 않을 것 같습니다. 재훈이는 가져가고 싶은 각 물건들의 부피와 얼마나 필요한지를 나타내는 절박도를 조사해 다음과 같은 목록을 만들었습니다.

물건	노트북 컴퓨터	카메라	XBOX365	커피그라인더	아령	백과사전
부피	4	2	6	4	2	10
절박도	7	10	6	7	5	4

캐리어의 용량이 정해져 있기 때문에 가져갈 수 있는 물건들의 부피 합은 캐리어의 용량 W 이하여야 합니다. 이때 절박도를 최대화할 수 있는 물건들의 목록을 계산하는 프로그램을 작성하세요.

Lec. 05. 동적 계획법 (2)

입력 입력의 첫 줄에는 테스트 케이스의 수 C (1≤C≤50)가 주어집니다. 각 테스트 케이스의 첫 줄에는 가져가고 싶은 물건의 수 N (1≤N≤100)과 캐리어의 용량 W (1≤W≤1000)가 주어집니다. 그 이후 N줄에 순서대로 각 물건의 정보가 주어집니다. 한 물건에 대한 정보는 물건의 이름, 부피, 절박도 순서대로 주어지며, 이름은 공백 없는 알파벳 대소문자 1글자 이상 20글자 이하의 문자열, 부피와 절박도는 1000 이하의 자연수입니다.

출력 각 테스트 케이스별 출력의 첫 줄에는 가져갈 수 있는 물건들의 최대 절박도 합과 가져갈 물건들의 개수를 출력합니다. 이후 한 줄에 하나씩 각 물건들의 이름을 출력합니다. 만약 절박도를 최대화하는 물건들의 조합이 여럿일 경우 아무 것이나 출력해도 좋습니다.

예제 입력

```
2
6 10
laptop 4 7
camera 2 10
xbox 6 6
grinder 4 7
dumbell 2 5
encyclopedia 10 4
6 17
laptop 4 7
camera 2 10
xbox 6 6
grinder 4 7
dumbell 2 5
encyclopedia 10 4
```

예제 출력

```
24 3
laptop
camera
grinder
30 4
laptop
camera
xbox
grinder
```

Lec. 05. 동적 계획법 (2)

배낭 문제: *Knapsack Problem*

배낭의 용량을 넘지 않으면서, 선택한 물건들의 가치의 합을 최대화하는 조합을 찾는 최적화 문제

INPUT:

물건의 집합: $S = [item_0, item_1, \cdots, item_{n-1}]$, 물건의 수: n, 배낭의 용량: W

물건의 무게: $w = [w_0, w_1, \cdots, w_{n-1}]$, 물건의 가치: $p = [p_0, p_1, \cdots, p_{n-1}]$

OUTPUT:

배낭에 담을 물건들의 부분 집합 $A \subseteq S$(최적해)와 그 가치의 총합 $\sum_{item_i \in A} p_i$(최적값)

GOAL:

배낭의 용량을 넘지 않으면서($\sum_{item_i \in A} w_i \leq W$), 가치의 합을 최대화($maximize \sum_{item_i \in A} p_i$)

배낭 문제의 두 가지 유형

분할 가능한 배낭 문제: *fractional knapsack problem*

물건을 분할하여 배낭에 일부만 담을 수 있는 조건: 예를 들어, 쌀 5kg 중 2kg만 담을 수 있다.

탐욕적 접근법으로 해결 가능:

단위 무게당 가치가 높은 물건부터 담고, 모두 담을 수 없는 아이템은 분할하여 담는다.

0-1 배낭 문제: 0-1 *knapsack problem*

물건을 배낭에 담거나/말거나 둘 중 하나인 조건: 예를 들어, 금괴 5kg을 담거나/말거나 둘 중 하나

탐욕적 접근법으로 최적해를 찾을 수 없음:

동적계획법이나 백트래킹을 통해 해결해야 하는 대표적인 NP-Hard 문제

0-1 배낭 문제를 완전 탐색(*DFS*)으로 무식하게 풀 수 있을까?

모든 경우의 수 탐색: 물건의 모든 조합을 하나하나 검사하고 이들 중 최적의 조합 찾기

각 물건에 대해 배낭에 담거나/말거나 두 가지의 선택이 있음

모든 경우의 수 = 멱집합($power\ set$)의 크기 = 2^n

$S = [item_0, item_1, item_2]$

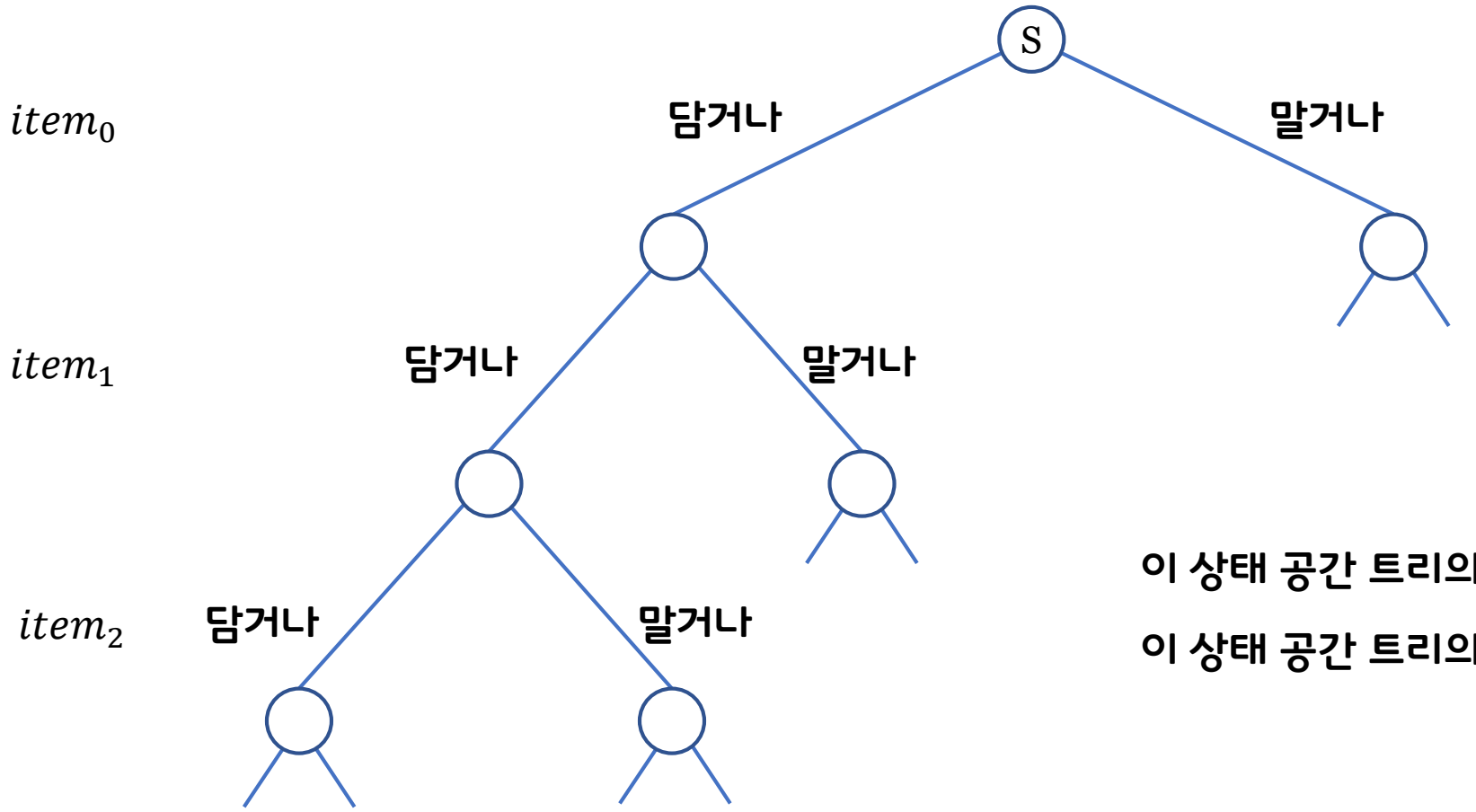
○ ○ ○

× × ×

$2 \times 2 \times 2 = 2^n$

$item_0$	$item_1$	$item_2$	A
×	×	×	$\emptyset$
○	×	×	$\{item_0\}$
×	○	×	$\{item_1\}$
○	○	×	$\{item_0, item_1\}$
×	×	○	$\{item_2\}$
○	×	○	$\{item_0, item_2\}$
×	○	○	$\{item_1, item_2\}$
○	○	○	$\{item_0, item_1, item_2\}$

완전 탐색(*DFS*)의 구현: 상태 공간 트리(*SST*)를 구상한다.



이 상태 공간 트리의 최대 깊이는?

이 상태 공간 트리의 노드 개수는?

Lec. 05. 동적 계획법 (2)

```
'''
P.5.1. 여행 짐 싸기 (ID: PACKING): 브루트-포스
'''
for _ in range(int(input())):
    n, W = map(int, input().split()) # 아이템의 수, 배낭 무게
    items = [input().split() for _ in range(n)] # 아이템 집합
    w = [int(items[i][1]) for i in range(n)] # 아이템의 무게
    p = [int(items[i][2]) for i in range(n)] # 아이템의 가치
    optval = 0 # 최적값
    optsol = [False] * n # 최적해
    packing(0, [False] * n) # 0번째 아이템부터 완전 탐색
    print(optval, sum(optsol))
    print(*[items[i][0] for i in range(n) if optsol[i]], sep = "\n")
```

$include[i]$: i번째 아이템을 배낭에 담을 건지 여부

Lec. 05. 동적 계획법 (2)

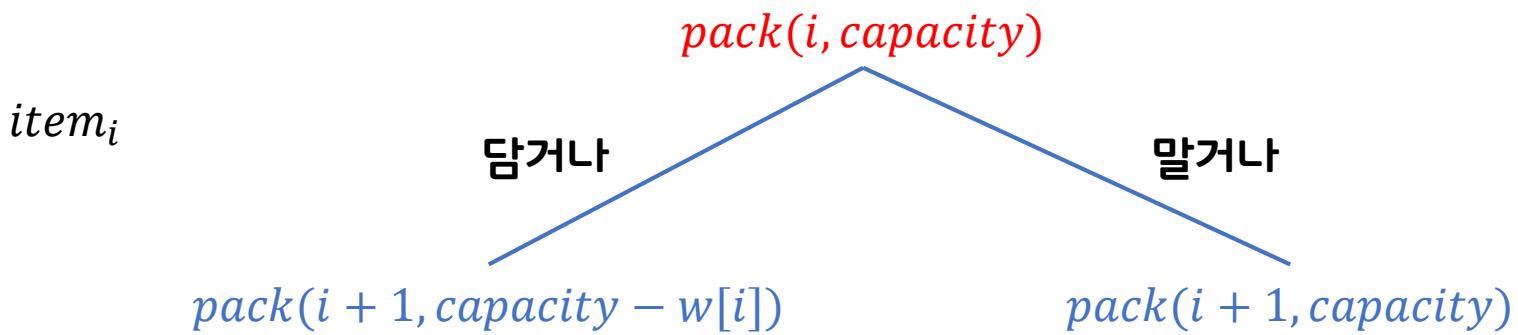
```
# 여행 짐싸기 문제를 완전탐색(DFS)으로 해결하기
def packing(i, include):
    global n, W, items, w, p, optval, optsol
    if i == n: # 모든 물건을 고려했으므로,
        weight = sum([w[j] for j in range(n) if include[j]]) # 담은 물건의 무게의 총합
        profit = sum([p[j] for j in range(n) if include[j]]) # 담은 물건의 가치의 총합
        if weight <= W and optval < profit: # 배낭 무게를 넘지 않고, 최적값이면
            optval = profit # 최적값을 업데이트
            optsol = include[:] # 최적해를 업데이트 (shallow copy)
    else:
        include[i] = True # i번 물건을 담고
        packing(i + 1, include) # 재귀 호출
        include[i] = False # i번 물건을 담지 않고
        packing(i + 1, include) # 재귀 호출
```

여행 짐싸기: 동적계획법(메모이제이션) 으로 해결하기

배낭의 남은 용량($capacity$)은

어떤 아이템을 담지 않기로 했다면, 현재 용량에 변함이 없지만,

담기로 했다면, 현재 용량에서 담은 아이템의 무게를 빼야 한다.



Lec. 05. 동적 계획법 (2)

$pack(i, capacity)$: 배낭 용량이 $capacity$만큼 남았을 때 i번째 이후 물건들로 얻을 수 있는 최대 이익

배낭에 물건을 담은 경우: $pack(i+1, capacity - w[i])$

배낭에 물건을 담지 않은 경우: $pack(i+1, capacity)$

$pack(i, capacity) = 0$: 만약 $capacity$가 0보다 작거나 같으면

$pack(i, capacity) = pack(i+1, capacity)$: 만약 $w[i]$가 $capacity$보다 크다면,

$pack(i, capacity) = \max\{pack(i+1, capacity), p[i] + pack(i+1, capacity - w[i])\}$: 아니면.

```
'''
P.5.1. 여행 짐 싸기 (ID: PACKING): 메모이제이션
'''
for _ in range(int(input())):
    n, W = map(int, input().split()) # 아이템의 수, 배낭 무게
    items = [input().split() for _ in range(n)] # 아이템 집합
    w = [int(items[i][1]) for i in range(n)] # 아이템의 무게
    p = [int(items[i][2]) for i in range(n)] # 아이템의 이익
    cache = [[-1] * (W + 1) for _ in range(n + 1)]
    optval = packing(0, W)
    optsol = reconstruct(0, W)
    print(optval, len(optsol))
    print(*optsol, sep = '\n')
```

Lec. 05. 동적 계획법 (2)

```python
# 여행 짐싸기 문제를 동적계획(메모이제이션)으로 해결하기
def packing(i, capacity):
    global n, w, p, cache
    if i == n or capacity <= 0:
        return 0
    elif cache[i][capacity] != -1:
        return cache[i][capacity]
    else:
        pick = packing(i + 1, capacity - w[i]) # 물건을 담은 경우
        drop = packing(i + 1, capacity) # 물건을 담지 않은 경우
        cache[i][capacity] = drop if capacity < w[i] else max(drop, p[i] + pick)
        return cache[i][capacity]
```

여행 짐싸기: 최적해의 재구성

$pack(i, capacity)$는 각 부분 문제에서 더 큰 값을 선택하므로,

별도의 $path$ 테이블에 선택을 저장하지 않고, 바로 역추적이 가능하다.

$pack(i, capacity) = pack(i+1, capacity)$인 경우:

i번째 아이템을 선택하지 않고도 최대 이익을 얻을 수 있으므로 아이템을 담지 않는다.

$pack(i, capacity) \neq pack(i+1, capacity)$인 경우:

i번째 아이템을 선택하는 것이 최대 이익을 얻을 수 있으므로 아이템을 담는다.

```python
# 여행 짐싸기 문제의 최적해를 재구성하기
def reconstruct(i, c):
    global n, items, w
    if i == n:
        return []
    elif packing(i, c) == packing(i + 1, c): # 담지 않아도 될 경우
        return reconstruct(i + 1, c)
    else: # 아니면 물건을 담는다.
        return [items[i][0]] + reconstruct(i + 1, c - w[i])
```

여행 짐싸기: 시간 복잡도 분석

완전 탐색(DFS)을 이용하는 방법:

상태 공간 트리의 노드 개수가 $O(2^n)$이므로 $O(2^n)$

동적계획법(메모이제이션)을 이용하는 방법:

아이템의 개수는 n이고 용량의 범위는 $[0, W]$이므로, 시간 복잡도는 $O(nW)$

만약, W의 값이 매우 큰 값이라고 해보자.

예) $W = n!$: $O(nW) = O(n \times n!) = O(n!)$

하지만, W의 값에 대한 탐색이 $O(2^n)$이상일 수는 없으므로, 전체 시간 복잡도는 $O(\min(2^n, nW))$

여행 짐싸기: 공간 복잡도 분석

앞서 구현한 알고리즘은 dp 테이블을 2차원 배열로 사용:

따라서, 공간 복잡도도 $O(\min(2^n, nW))$

공간 복잡도 자체를 개선하기는 어렵지만,

딕셔너리를 사용해서 불필요한 저장 공간의 사용을 없앨 수 있다.

```
for i in range(n):
    print(*dp[i])
```

```
-1 -1 -1 -1 -1 -1 -1 -1 -1 -1 24
-1 -1 -1 -1 -1 -1 17 -1 -1 -1 22
-1 -1 -1 -1  7 -1 12 -1 12 -1 13
-1 -1  5 -1  7 -1 12 -1 12 -1 12
-1 -1  5 -1  5 -1  5 -1  5 -1  5
-1 -1  0 -1  0 -1  0 -1  0 -1  4
```

Lec. 05. 동적 계획법 (2)

```
# 여행 짐싸기 문제를 딕셔너리를 이용한 메모이제이션으로 해결하기
def packing(i, capacity):
    global n, w, p, cache                    cache = {} # 메모리의 효율적 사용을 위한 딕셔너리
    if i == n or capacity <= 0:
        return 0
    elif (i, capacity) in cache: # 이미 계산한 경우 확인
        return cache[(i, capacity)]
    else:
        pick = packing(i + 1, capacity - w[i]) # 물건을 담은 경우
        drop = packing(i + 1, capacity) # 물건을 담지 않은 경우
        cache[(i, capacity)] = drop if capacity < w[i] else max(drop, p[i] + pick)
        return cache[(i, capacity)]
```

여행 짐싸기: 태뷸레이션으로 풀면?

```python
# 여행 짐싸기 문제를 태뷸레이션으로 해결하기
def packing(n, W, w, p):
    dp = [[0] * (W + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for j in range(1, W + 1):
            if w[i - 1] > j:
                dp[i][j] = dp[i - 1][j]
            else:
                dp[i][j] = max(dp[i - 1][j], p[i - 1] + dp[i - 1][j - w[i - 1]])
    return dp[n][W], dp
```

시간 복잡도와 공간 복잡도가 모두 $O(nW) \geq O(\min(2^n, nW))$

$n = 10, W = 10^{10}$ **이면?**

0-1 배낭 문제를 푸는 다른 방법:

계속 진행할 경우의 한계 이익(*bound*)으로 가지치기를 하기 쉬워진다.

백트래킹(*backtracking*)을 이용하는 방법:

단위 무게당 이익의 내림차순으로 정렬한 후,

DFS로 탐색을 하면서 가지치기(*pruning*)를 한다.

분기한정법(*Branch-and-Bound*)을 이용하는 방법:

단위 무게당 이익의 내림차순으로 정렬한 후,

Best FS(*Priority Queue*를 사용하는 BFS)로 탐색을 하면서 가지치기를 한다.

AL2.5.2: 최대 증가 부분 수열 (ID: LIS)

문제

어떤 정수 수열에서 0개 이상의 숫자를 지우면 이 수열의 부분 수열 (*subsequence*) 를 얻을 수 있다. 예를 들어 [10, 7, 4, 9] 의 부분 수열에는 [7, 4, 9], [10, 4], [10, 9] 등이 있다. 단, [10, 4, 7]은 원래 수열의 순서와 다르므로 [10, 7, 4, 9]의 부분 수열이 아니다.

어떤 부분 수열이 순 증가할 때 이 부분 수열을 증가 부분 수열 (*increasing subsequence*) 라고 한다. 주어진 수열의 증가 부분 수열 중 가장 긴 것의 길이를 계산하는 프로그램을 작성하라.

어떤 수열의 각 수가 항상 이전의 수보다 클 때, 이 수열을 순증가(*strictly increasing*)한다고 한다. 두 인접한 숫자가 같을 수도 있을 때는 수열이 단조 증가(*monotonically increasing*)한다고 한다.

Lec. 05. 동적 계획법 (2)

입력 입력의 첫 줄에는 테스트 케이스의 수 C (<= 50) 가 주어진다. 각 테스트 케이스의 첫 줄에는 수열에 포함된 원소의 수 N (<= 500) 이 주어진다. 그 다음 줄에 수열이 N개의 정수가 주어진다. 각 정수는 1 이상 100,000 이하의 자연수이다.

출력 각 테스트케이스마다 한 줄씩, 주어진 수열의 가장 긴 증가 부분 수열의 길이를 출력한다.

예제 입력

```
3
4
1 2 3 4
8
5 4 3 2 1 6 7 8
8
5 6 7 8 1 2 3 4
```

예제 출력

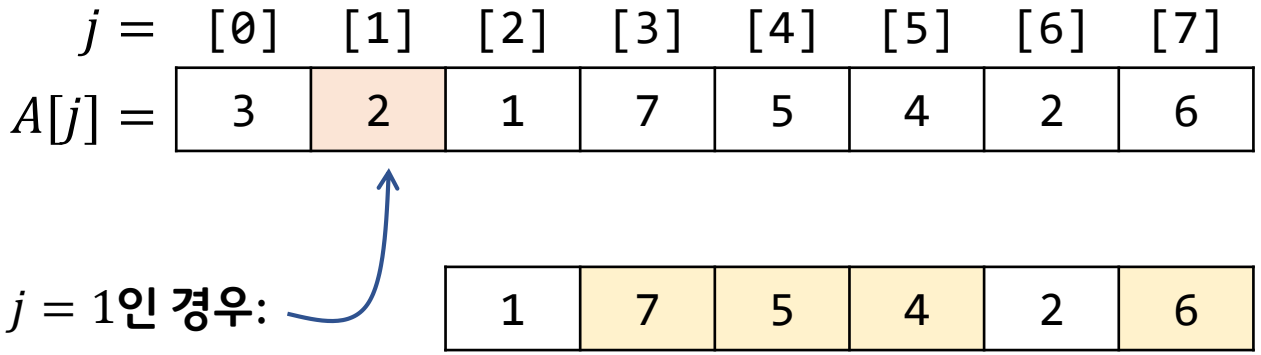
```
4
4
4
```

완전 탐색(재귀)에서 시작하기

숫자 하나씩으로 조각낸 뒤, 한 조각에서 숫자 하나씩을 선택하는 완전 탐색(재귀) 전략

$lis(A)$: 수열 A의 모든 증가 부분 수열 중에서 가장 긴 것의 길이

수열 A의 첫 번째 수를 고르고 나머지 부분을 재귀 호출



부분 수열 $B = [7,5,4,6]$: $A[1]$보다 큰 수 들의 부분 수열

$lis(B)$를 재귀 호출: 반환된 LIS를 $A[1]$ 뒤에 붙이면 $A[1]$로 시작하는 LIS

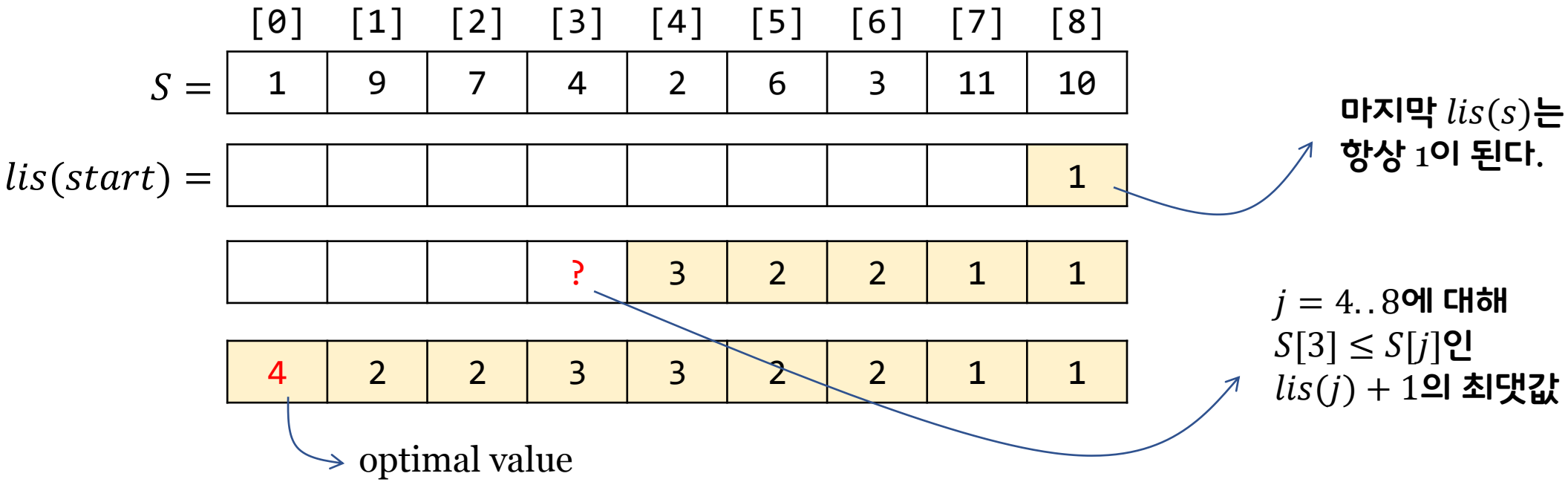
```
'''
P.5.2. 최대 증가 부분 수열 (ID: LIS)
'''
for _ in range(int(input())):
    n = int(input()) # 원소의 수
    A = list(map(int, input().split())) # n개의 정수
    print(lis(n, A))
```

```python
# LIS 문제를 해결하는 완전 탐색 알고리즘
def lis(n, A):
    if not A: # A가 빈 수열일 때
        return 0
    else:
        ret = 0
        for i in range(n):
            B = [] # A[i] 뒤에 있는 A[i] 보다 큰 수들의 부분 수열
            for j in range(i + 1, n):
                if A[i] < A[j]:
                    B.append(A[j])
            ret = max(ret, 1 + lis(len(B), B))
        return ret
```

LIS 문제를 동적계획법(메모이제이션)으로 풀기

$lis(start)$: $S[start]$에서 시작하는 부분 증가 수열 중 최대의 길이

$$lis(start) = \max_{next=[start+1,n-1]} (list(next) + 1), \text{ where } S[start] < S[next]$$



Lec. 05. 동적 계획법 (2)

```
# LIS 문제를 해결하는 메모이제이션 알고리즘
def lis(start):
    global n, S, cache
    if cache[start] != -1: # 이미 계산된 값이 있으면 재활용
        return cache[start]
    else:
        ret = 1 # 항상 S[start]는 존재하므로 길이는 최하 1
        for next in range(start + 1, n):
            if S[start] < S[next]:
                ret = max(ret, lis(next) + 1)
        cache[start] = ret # 결과값을 캐시에 저장
        return cache[start]

cache = [-1] * n
print(max(lis(i) for i in range(n)))
```

LIS 문제를 동적계획법(태뷸레이션)으로 풀기

$$dp[i] = \max_{j=[i+1,n-1]}(dp[j]+1), \text{ where } S[i] < S[j]$$

```
# LIS 문제를 해결하는 동적계획법(태뷸레이션) 알고리즘
def lis(n, S):
    dp = [1] * n # dp 테이블의 초깃값은 1
    for i in range(n - 1, -1, -1): # bottom-up
        for j in range(i + 1, n):
            if S[i] < S[j]:
                dp[i] = max(dp[i], dp[j] + 1)
    return max(dp) # dp 테이블의 최댓값이 LIS의 길이
```

LIS 문제: 최적값이 아니라, 최적해를 구하기

```
# 최적값을 찾는 과정을 choices에 기록한다.
def lis(n, S):
    dp = [1] * n
    choices = [-1] * n # 선택 과정을 기록하는 path 테이블

    for i in range(n - 1, -1, -1):
        for j in range(i + 1, n):
            if S[i] < S[j] and dp[i] < dp[j] + 1:
                dp[i] = dp[j] + 1
                choices[i] = j # 다음 선택의 인덱스 저장

    start = max(range(n), key=lambda i: dp[i])

    return dp[start], start, choices
```

```
for _ in range(int(input())):
    n = int(input())
    S = list(map(int, input().split()))
    optval, start, choices = lis(n, S)
    optsol = reconstruct(start)
    print(optval)
    print(*optsol)
```

```
# choices로부터 최적해를 재구성한다.
def reconstruct(start):
    global n, S, choices
    if start == -1: # 종료 조건: 다음 선택은 없다.
        return []
    else:
        return [S[start]] + reconstruct(choices[start])
```

이보다 더 효율적으로 풀 수 있을까?: 이분탐색을 이용한 $O(n \lg n)$ 알고리즘

텅 빈 수열에서 시작해 숫자를 하나씩 추가해 나가며

각 길이를 갖는 증가 수열 중 가장 마지막 수가 제일 작은 것은 무엇인지를 추적하는 전략

$S =$

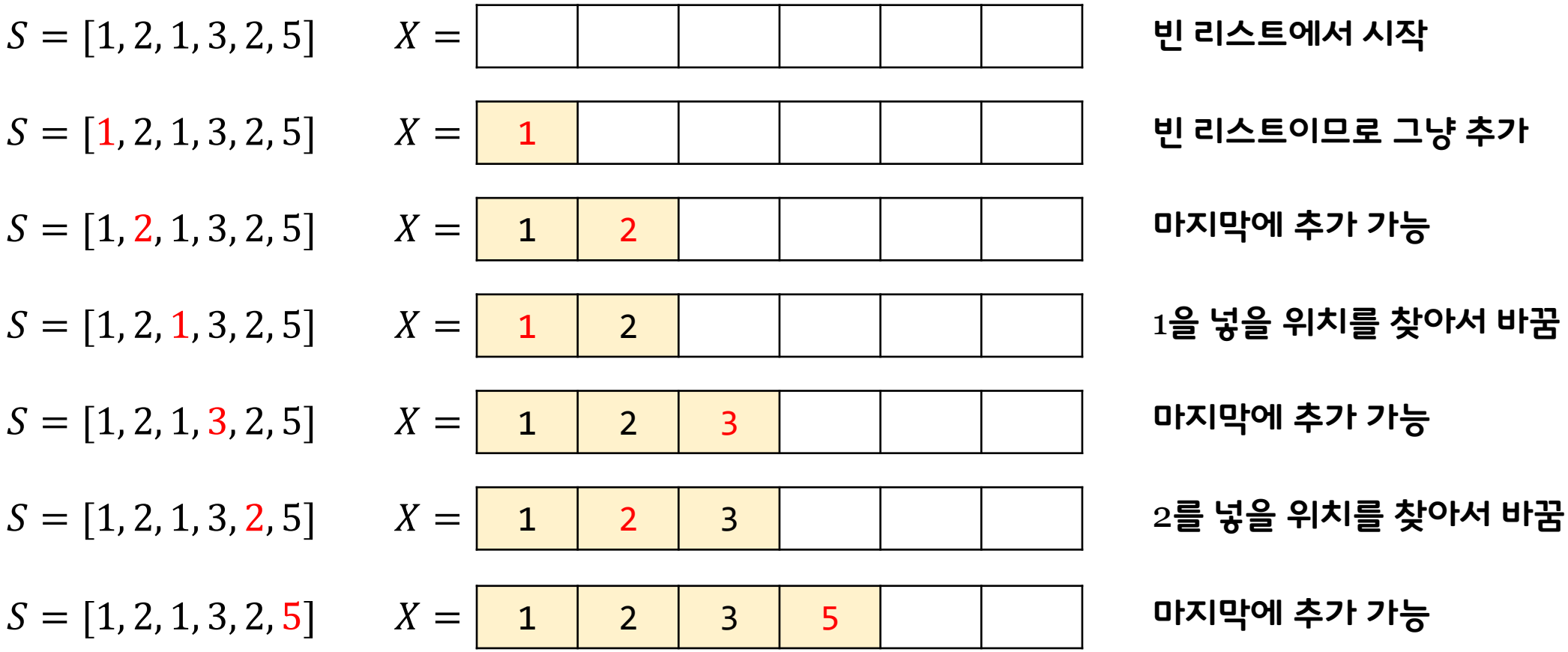
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
5	6	7	1	2	…			

길이가 3인 부분증가수열: [5, 6, 7]

길이가 2인 부분증가수열: [5, 6], [5, 7], [6, 7], [1, 2]

끝 숫자가 작을 수록 더 긴 LIS를 만들 수 있다: 이 중에서 [1, 2]가 가장 유리하다.

$X[i]$: 지금까지 만든 부분 배열이 갖는 길이 i인 증가 부분 수열 중 최소인 마지막 값



Lec. 05. 동적 계획법 (2)

동적계획법 + 이분 탐색:

동적계획: X **배열은** LIS**를 구성하기 위해 현재까지의 가장 작은 마지막 값들로 구성된 배열**

이분탐색: X**는 항상 정렬된 상태로 유지되므로 이분 탐색을 할 수 있음**

$S[i]$**가** $X[-1]$**보다 크면:**

X**의 마지막에** $S[i]$**를 추가**

$S[i]$**가** $X[-1]$**보다 크지 않으면:**

들어갈 위치를 찾아서(Binary Search) **해당 위치에 있는 원소를** $S[i]$**로 교체**

Lec. 05. 동적 계획법 (2)

```python
from bisect import bisect_left

# LIS 문제를 이분 탐색을 이용하여 해결하는 알고리즘
def lis(n, S):
    X = [S[0]] # X[i] 길이가 i인 증가 부분 수열의 마지막 값 중에서 가장 작은 값
    for k in range(1, n):
        if S[k] > X[-1]: # X의 마지막 원소보다 크면
            X.append(S[k]) # 배열의 마지막에 추가
        else:
            # 추가할 위치를 이분 탐색해서 k로 대체
            X[bisect_left(X, S[k])] = S[k]
    return len(X), X
```

X 자체가 실제 LIS는 아니지만,
각 원소의 이전 위치를 별도록 기록하여 LIS를 구할 수 있다.

AL2.5.3: k번째 최대 증가 부분 수열 (ID: KLIS)

문제

어떤 정수 수열에서 0개 이상의 숫자를 지우면 이 수열의 부분 수열 (*subsequence*) 를 얻을 수 있다. 예를 들어 10 7 4 9 의 부분 수열에는 7 4 9, 10 4, 10 9 등이 있다. 단, 10 4 7 은 원래 수열의 순서와 다르므로 10 7 4 9 의 부분 수열이 아니다.

어떤 부분 수열이 단조 증가할 때 이 부분 수열을 증가 부분 수열 (*increasing subsequence*) 라고 하며, 이 중 가장 긴 것을 최대 증가 부분 수열 (LIS, *longest increasing subsequence*) 라고 한다. 예를 들어, 5 20 21 22 8 9 10 의 최대 증가 부분 수열은 5 8 9 10 이다.

어떤 수열에는 LIS 가 두 개 이상 있을 수 있다. 예를 들어, 4 5 6 1 2 3 의 LIS 는 두 개가 있다.

모든 숫자가 서로 다른 (중복 숫자가 없는) 수열이 주어질 때, 이 수열의 LIS 중 사전 순서대로 맨 앞에서 k번째 있는 LIS 를 출력하는 프로그램을 작성하시오.

Lec. 05. 동적 계획법 (2)

입력 입력의 첫 줄에는 테스트 케이스의 수 C (<= 50) 가 주어진다. 각 테스트 케이스의 첫 줄에는 수열에 포함된 원소의 수 N (<= 500) 과 K 가 주어진다. K 는 32비트 부호 있는 정수에 저장할 수 있다. 그 다음 줄에 N개의 정수로 수열이 주어진다. 각 정수는 1 이상 100,000 이하이며, 같은 수는 두 번 등장하지 않는다.

주어진 수열의 LIS 는 최소 K 개 있다고 가정해도 좋다.

출력 각 테스트케이스마다 두 줄을 출력한다. 첫 줄에는 LIS 의 길이 L 을 출력하고, 그 다음 줄에 L 개의 정수로 K번째 LIS 를 출력한다.

예제 입력

```
3
9 2
1 9 7 4 2 6 3 11 10
8 4
2 1 4 3 6 5 8 7
8 2
5 6 7 8 1 2 3 4
```

예제 출력

```
4
1 2 3 11
4
1 3 6 8
4
5 6 7 8
```

최적화 문제에서 k번째 답을 구하는 문제를 푸는 레시피

1. 최적화 문제의 최적값을 구하는 문제를 푼다.
2. 최적화 문제의 최적해를 세는 문제를 푼다.
3. 최적해의 수를 기반으로 최적해를 재구성한다.

```
'''
P.5.3. k번째 최대 증가 부분 수열 (ID: KLIS)
'''
MAX = 2000000001
for _ in range(int(input())):
    n, k = map(int, input().split()) # 원소의 수, k번째
    S = list(map(int, input().split())) # 수열
    klis(n, k, S)
```

```python
# k번째 LIS 문제를 해결하기
def klis(n, k, S):
    global dp_len, dp_cnt
    S += [-float('inf')] # 편의를 위해 마지막에 -inf 추가: S[-1] = -inf
    dp_len = make_length(n, S) # LIS의 길이 DP
    dp_cnt = make_count(n, S) # LIS의 개수 DP
    optsol = []
    reconstruct(-1, k - 1, optsol)
    print(dp_len[-1] - 1) # -inf의 LIS에서 1을 뺀 값이 최적값
    print(*optsol[1:]) # 최적해에서 -inf를 제외하고 출력
```

LIS의 개수 세기

$S[start]$에서 시작하는 LIS의 수는

최대치를 반환하는 $S[next]$에서 시작하는 LIS의 개수와 같다.

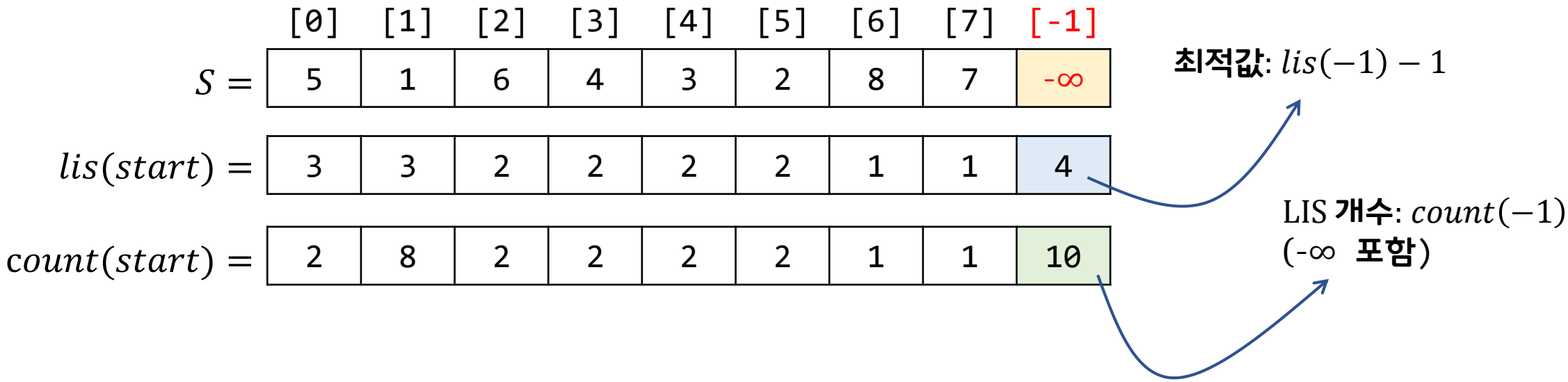
$count(i)$: i번째 위치에서 시작하는 LIS의 개수

$count(i) = \sum_{j=i+1}^{n-1} count(j)$, where $S[i] < S[j]$ and $lis(i) = lis(j) + 1$

LIS의 시작 위치 고정하기

dp 테이블에서 LIS의 시작 위치가 다르면 길이와 개수를 찾기가 불편하기 때문에

$S[-1] = -\infty$라고 가정하면 LIS의 시작 위치가 항상 -1에서 시작하도록 고정시킬 수 있다.



```python
# LIS의 길이를 저장하는 DP 테이블 생성
def make_length(n, S):
    dp = [1] * (n + 1) # -inf를 추가했으므로 크기는 n+1
    for i in range(n - 1, -2, -1): # bottom-up (-1까지)
        for j in range(i + 1, n):
            if S[i] < S[j]:
                dp[i] = max(dp[i], dp[j] + 1)
    return dp
```

```
# LIS의 개수를 저장하는 DP 테이블 생성
def make_count(n, S):
    global dp_len
    dp = [1] * (n + 1) # -inf를 추가했으므로 크기는 n+1
    for i in range(n - 1, -2, -1): # bottom-up (-1까지)
        if dp_len[i] == 1: # LIS의 길이가 1이면 개수도 1
            dp[i] = 1
        else: # 아니면 조건에 부합하는 dp[j]의 값을 모두 더한다.
            ret = 0
            for j in range(i + 1, n):
                if S[i] < S[j] and dp_len[i] == dp_len[j] + 1:
                    ret = min(MAX, ret + dp[j])
            dp[i] = ret
    return dp
```

LIS의 재구성: k번째 최적해 찾기

$skip = k - 1$로 시작

$skip$개의 LIS를 건너 뛰고 사전순으로 가장 앞에 오는 LIS를 찾아야 한다.

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[-1]
$S =$	5	1	6	4	3	2	8	7	-∞
$lis(start) =$	3	3	2	2	2	2	1	1	4
$count(start) =$	2	8	2	2	2	2	1	1	10

$skip = 5$인 경우: LIS는 반드시 $S[-1] = -\infty$에서 시작한다. $klis = [-\infty]$

$S[-1]$ 뒤에 올 수 있는 숫자 중에서 가장 작은 수는 $S[1] = 1$이다.

$S[1]$로 시작하는 LIS의 개수는 8개이므로 $skip$보다 크다.

따라서, $S[-1]$ 뒤에는 $S[1]$이 와야 한다. $klis = [-\infty, 1]$

Lec. 05. 동적 계획법 (2)

$S[1]$ 뒤에 올 수 있는 숫자 $S[2], S[3], S[4], S[5]$ 를 사전순으로 줄을 세워 보면,
이들의 첫 번째 수는 모두 1이기 때문에, 두 번째 숫자에 따라 순서가 결정된다.

skip = 5 → $1, 2, \ldots$ / $1, 2, \ldots$ } $count(5) = 2$

skip = 3 → $1, 3, \ldots$ / $1, 3, \ldots$ } $count(4) = 2$

skip = 1 → $1, 4, \ldots$ / $1, 4, \ldots$ } $count(3) = 2$

$1, 6, \ldots$ / $1, 6, \ldots$ } $count(2) = 2$

Lec. 05. 동적 계획법 (2)

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[-1]
$S =$	5	1	6	4	3	2	8	7	-∞
$lis(start) =$	3	3	2	2	2	2	1	1	4
$count(start) =$	2	8	2	2	2	2	1	1	10

$skip = 5$: $S[1]$ 뒤에 올 수 있는 숫자는? $[6, 4, 3, 2]$

다음 두 가지 조건을 만족해야 한다: $S[i] < S[j], lis(i) = lis(j) + 1$

$S[1]$ 뒤에 올 수 있는 숫자 중에서 가장 작은 수는 $S[5] = 2$이다.

$S[5]$로 시작하는 LIS의 개수는 2개이므로 $skip$보다 작다.

따라서, $S[1]$ 뒤에 $S[5]$는 올 수 없으므로 2개를 건너 뛴다. $skip = 3$

마찬가지로, $S[1]$ 뒤에 $S[4]$는 올 수 없으므로 2개를 건너 뛴다. $skip = 1$

$S[3]$으로 시작하는 LIS의 개수가 2개이므로 $skip$보다 크다.

$klis = [-\infty, 1, 4]$

Lec. 05. 동적 계획법 (2)

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[-1]
$S =$	5	1	6	4	3	2	8	7	-∞
$lis(start) =$	3	3	2	2	2	2	1	1	4
$count(start) =$	2	8	2	2	2	2	1	1	10

$skip = 1$: $S[3]$ 뒤에 올 수 있는 숫자는? $[7, 8]$

다음 두 가지 조건을 만족해야 한다: $S[i] < S[j], lis(i) = lis(j) + 1$

$S[3]$ 뒤에 올 수 있는 숫자 중에서 가장 작은 수는 $S[7] = 7$이다.

$S[7]$로 시작하는 LIS의 개수는 1개이므로 $skip$과 같다.

따라서, $S[3]$ 뒤에 $S[7]$은 올 수 없으므로 1개를 건너 뛴다. $skip = 0$

$S[8]$로 시작하는 LIS의 개수가 1개이므로 $skip$보다 크다.

$klis = [-\infty, 1, 4, 8]$

Lec. 05. 동적 계획법 (2)

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[-1]
$S =$	5	1	6	4	3	2	8	7	-∞
$lis(start) =$	3	3	2	2	2	2	1	1	4
$count(start) =$	2	8	2	2	2	2	1	1	10

$skip = 0$: $S[6]$ 뒤에 올 수 있는 숫자는? 없다.

$klis = [-\infty, 1, 4, 8]$

Lec. 05. 동적 계획법 (2)

```
# S[start]에서 시작하는 LIS 중 사전순으로 skip개 건너뛴 수열을 result에 저장
def reconstruct(start, skip, result):
    global n, S, dp_len, dp_cnt
    result.append(S[start]) # 1. S[start]는 항상 LIS에 포함
    followers = [] # 2. 뒤에 오는 숫자들과 위치의 목록을 만든다. (숫자, 위치)
    for next in range(start + 1, n):
        if S[start] < S[next] and dp_len[start] == dp_len[next] + 1:
            followers.append((S[next], next))
    followers.sort() # 숫자의 오름차순으로 정렬
    for f in followers: # 3. k번째 LIS의 다음 숫자를 찾는다.
        cnt = dp_cnt[f[1]] # 이 숫자를 뒤에 이어서 만들 수 있는 LIS의 개수를 본다.
        if cnt <= skip: # 건너 뛰어야 할 개수이면
            skip -= cnt # 건너 뛰자.
        else: # 4. 아니면 다음 위치 f[1]에서 재귀 호출
            reconstruct(f[1], skip, result)
            break
```

Any Questions?

